Trabajo Práctico Integrador

Metodología de Sistemas II

Evolución de un sistema de solicitudes académicas: calidad interna, deuda técnica, refactoring y revisión técnica.

Dato	Información
Alumno	Bagni Mattia
Modalidad	Individual, con autorización informada por el alumno
Comisión	Pendiente de completar
Docente	Pendiente de completar
Fecha de entrega	Pendiente de completar
Estado del documento	Borrador para revisión y defensa del alumno

Resumen ejecutivo

Se construyó un prototipo local de solicitudes con Angular, Django y SQLite.

La base recibida contenía plantillas, sin operaciones académicas implementadas.

Sobre la primera versión funcional se diagnosticaron diez debilidades técnicas.

Se separaron responsabilidades y se aplicaron siete mejoras de estructura.

Las pruebas conservaron las respuestas y el listado pasó de 22 consultas a 1.

El desarrollo contó con asistencia sustancial de IA, declarada en este informe.

Pendientes para la entrega: revisión personal, ejecución en el equipo del alumno, datos de portada, publicación del repositorio, video propio y reflexión individual. No se presentan esos puntos como realizados.

1. Sistema base y alcance

El caso es un sistema de gestión de solicitudes académicas. Permite registrar un pedido de un estudiante, indicar tipo y prioridad, asignar un responsable, cambiar el estado y consultar un reporte. El alcance se mantiene acotado para concentrar el análisis en la mantenibilidad.

Actores y entidades

El estudiante origina una solicitud. Un operador de secretaría la registra y actualiza su seguimiento. Un responsable representa el área que la atiende. En esta demo los roles describen el dominio; no se implementó autenticación ni permisos por usuario.

Entidad	Datos principales	Relación
Estudiante	Nombre y legajo ficticios	Un estudiante puede tener muchas solicitudes.
Responsable	Nombre y área ficticios	Puede atender muchas solicitudes; la asignación es opcional.
Solicitud	Título, descripción, tipo, prioridad y estado	Pertenece a un estudiante y puede tener un responsable.

Operaciones verificables

Operación	API y comportamiento
Registrar	POST /api/solicitudes/: valida y guarda; devuelve 201.
Consultar y filtrar	GET /api/solicitudes/: título, estado y prioridad combinables.
Cambiar estado	PATCH /api/solicitudes/<id>/: pendiente, en_proceso o resuelta.
Asignar responsable	PATCH en la misma ruta; null permite quitar la asignación.
Reportar	GET /api/reporte/: total y cantidad por estado, sin filtros.
Catálogos	GET /api/catalogos/: datos ficticios y opciones; entrega cookie CSRF.

Tecnología y almacenamiento

Se utilizó Angular 22.2 en el frontend y Python con Django 4.2.30 en el backend, conservando las dependencias aportadas. SQLite guarda datos en db.sqlite3 dentro del equipo local. El archivo se genera con las migraciones y no se distribuye una base recibida con posibles datos personales.

La interfaz incluye formulario, listado, filtros, asignación y reporte. Las pruebas de componentes verifican el DOM en JSDOM. La integración se probó por HTTP real; no se dispone de una captura de navegador y no se adjuntan imágenes simuladas.

2. Calidad externa e interna

Perspectiva	Observación sobre la versión funcional inicial
Calidad externa	Permite completar las operaciones esperadas, informa errores y conserva los datos al consultar. Se verificó con pruebas, no solo con una descripción de funcionalidades.
Calidad interna	Las vistas concentran decisiones, repiten el mapeo público y cargan relaciones por cada fila. El componente Angular mezcla transporte HTTP con el estado de la pantalla.
Consecuencia	La corrección visible no garantiza bajo costo de cambio. Un nuevo campo podría exigir cambios en tres respuestas y un listado más grande aumenta el número de consultas.

Origen y versiones del caso

Estado	Contenido y autoría del proceso
Archivo recibido	Dos esqueletos Django y Angular con la pantalla generada. Sin entidades ni operaciones académicas. Se conserva el ZIP original sin modificar.
estado-inicial	Primera versión funcional construida después de recibir las plantillas, con asistencia sustancial de ChatGPT. No se presenta como implementación previa del alumno.
estadorefactorizado	Mismas operaciones, con responsabilidades separadas, contrato tipado y consultas optimizadas. El bundle conserva los cambios reales de esta sesión.

Criterio de análisis

El diagnóstico usa evidencia observable: una función que reúne varias responsabilidades, código duplicado, un conjunto de parámetros relacionado y consultas ejecutadas en el listado. No se asignaron costos monetarios ni métricas de cobertura que no se hayan medido.

La prioridad alta corresponde a cambios que afectan varias rutas o escalan con el volumen; la media, a inconsistencias y errores de edición; la baja, a una mejora localizada. Las ubicaciones del diagnóstico se refieren al tag estado-inicial. Los archivos finales tienen otras líneas.

Trazabilidad

docs/origen.json registra el nombre y SHA-256 del ZIP recibido. docs/diagnostico.json conserva el archivo, la línea y un fragmento de cada hallazgo. La comparación completa puede revisarse con git diff estado-inicial estadorefactorizado.

3. Deuda técnica y code smells

Los diez hallazgos siguientes pertenecen a la primera versión funcional. Las primeras cinco debilidades explican cambios propensos a inconsistencias. La tabla continúa en la página siguiente.

D01. Vista con varias responsabilidades | Prioridad alta

Ubicación: backend/mi_app/views.py:26 (estado-inicial).

Evidencia: solicitudes combina alta, filtros, consultas y respuesta. Impacto y riesgo: Un ajuste local obliga a revisar varias rutas del flujo.

D02. Parseo de JSON duplicado | Prioridad media

Ubicación: backend/mi_app/views.py:29 (estado-inicial).

Evidencia: Las dos vistas de escritura repiten lectura y comprobación del cuerpo. Impacto y riesgo: Puede divergir el manejo de entradas y mensajes de error.

D03. Mapeo público duplicado | Prioridad alta

Ubicación: backend/mi_app/views.py:51 (estado-inicial).

Evidencia: El diccionario de Solicitud se copia en alta, listado y actualización. Impacto y riesgo: Un campo nuevo puede aparecer solo en algunas respuestas.

D04. Lista de parámetros extensa | Prioridad media

Ubicación: backend/mi_app/views.py:20 (estado-inicial).

Evidencia: crear_solicitud recibe seis argumentos posicionales. Impacto y riesgo: Es fácil intercambiar argumentos de tipo string o perder contexto.

D05. Literales repetidos | Prioridad media

Ubicación: backend/mi_app/views.py:40 (estado-inicial).

Evidencia: Estados, tipos y prioridades se repiten además de sus elecciones en el modelo. Impacto y riesgo: Una modificación puede dejar reglas incompatibles.

3. Diagnóstico (continuación)

D06. N+1 en acceso a relaciones | Prioridad alta

Ubicación: backend/mi_app/views.py:71 (estado-inicial).

Evidencia: El bucle accede a estudiante.nombre y responsable.nombre con carga diferida. Impacto y riesgo: 22 consultas para 11 solicitudes en el escenario medido.

D07. Conteos repetidos | Prioridad baja

Ubicación: backend/mi_app/views.py:113 (estado-inicial).

Evidencia: El reporte ejecuta total y tres count separados. Impacto y riesgo: Costo evitable y estados escritos manualmente.

D08. Acoplamiento de pantalla a HTTP | Prioridad alta

Ubicación: frontend/src/app/app.ts:8 (estado-inicial).

Evidencia: App conoce URLs, parámetros, formulario y avisos. Impacto y riesgo: Modificar el transporte obliga a editar la pantalla.

D09. Contrato sin tipos precisos | Prioridad media

Ubicación: frontend/src/app/app.ts:9 (estado-inicial).

Evidencia: Catálogos, reporte y lista usan any. Impacto y riesgo: Errores de estructura se detectan al ejecutar.

D10. Operaciones persistentes en la vista | Prioridad alta

Ubicación: backend/mi_app/views.py:81 (estado-inicial).

Evidencia: La vista de actualización asigna campos y guarda. Impacto y riesgo: Las decisiones quedan ligadas al transporte HTTP.

Deuda y decisión de corrección

Se priorizó separar responsabilidades, centralizar el contrato y evitar N+1. Las demás mejoras se aplicaron en pasos verificables. No toda repetición justifica una abstracción: no se incorporaron repositorios genéricos, microservicios ni jerarquías sin una necesidad del caso.

4. SOLID y Clean Code

La aplicación de principios se justifica por problemas concretos. No se presenta una lista de siglas como si todas hubieran requerido cambios en este prototipo.

Principio	Aplicación y límite
SRP	Aplicado: vistas para HTTP; validación para entradas; servicios para operaciones; consultas para lectura; presentación para el JSON público. El frontend delega HTTP en SolicitudesApi.
OCP	Mejora parcial: las elecciones del modelo centralizan opciones. Un nuevo tipo todavía puede requerir modificar uniones TypeScript. No se afirma extensión sin cambios en todo el sistema.
LSP	No se añadieron jerarquías propias. No corresponde inventar una demostración de sustitución ni forzar herencia solo para mencionar el principio.
ISP	Se mantienen contratos pequeños por operación. No se construyó una interfaz general ni se atribuye una aplicación formal completa a esta separación.
DIP	Aplicación parcial en Angular: inyección del servicio y HttpClient, sustituible en pruebas. El backend conserva dependencia directa del ORM de Django; no implementa inversión total.

Responsabilidades finales

Archivo o módulo	Responsabilidad
views.py	Coordinar entrada y salida HTTP.
validacion.py + dtos.py	Validar el formato y representar datos relacionados.
servicios.py	Registrar y actualizar solicitudes dentro de transacciones.
consultas.py	Consultar relaciones, filtrar y agregar conteos.
presentacion.py	Leer JSON, mapear la solicitud y responder errores previstos.
solicitudes-api.ts	Encapsular rutas, parámetros y métodos del backend.
app.ts + modelos.ts	Estado de la pantalla y tipos del contrato.

Clean Code

Se utilizan nombres del dominio, funciones enfocadas, DTOs inmutables, tipado estricto y un único mapeo de salida. Los errores esperados se distinguen de fallas inesperadas. No se agregó un catch general que oculte defectos. Se mantuvo el esquema de datos para reducir variables durante la comparación.

5. Refactoring seguro

R01. Extract Method: lectura del cuerpo

Antes se repetía json.loads y la comprobación del objeto. Después leer_json concentra esa lectura y errores_json transforma únicamente errores previstos en respuestas. Se conservan los mensajes y códigos de las entradas probadas.

```
Antes: datos = json.loads(request.body)  # en dos vistas
Después: datos = leer_json(request)
```

R02. Move Method: operaciones de escritura

Las vistas dejan de actualizar directamente los campos. servicios.registrar_solicitud y servicios.actualizar_solicitud reúnen las operaciones y su transacción. La vista solo valida, delega y presenta el resultado.

```
Antes, en la vista: s.estado = datos["estado"]; s.save()
Después: solicitud = servicios.actualizar_solicitud(pk, cambios)
```

R03. Introduce Parameter Object

NuevaSolicitud agrupa los seis datos relacionados. La dataclass es inmutable y sus nombres hacen explícito el significado de cada valor. Se reduce la posibilidad de intercambiar argumentos posicionales.

```
Antes: crear_solicitud(eid, titulo, descripcion, tipo, prioridad, rid)
Después: registrar_solicitud(datos: NuevaSolicitud)
```

R04. Extract Method: respuesta pública

El diccionario repetido en tres lugares se reemplaza por solicitud_a_dict. Alta, consulta y actualización mantienen el mismo contrato sin copiar su definición.

```
Antes: {"id": s.id, "titulo": s.titulo, ...}  # tres copias
Después: solicitud_a_dict(solicitud)
```

Protección del comportamiento

Las 21 pruebas iniciales del backend y las 6 iniciales del frontend se ejecutaron antes y después. También se capturaron diez pasos de API en una base temporal y se compararon status y JSON. Las capturas no incluyen fechas variables ni tokens.

5. Otras mejoras y verificación

Cambio	Antes y después	Evidencia
R05. Mover acceso HTTP	App conoce rutas y parámetros; después usa SolicitudesApi inyectado y tipos explícitos.	Las seis pruebas iniciales de pantalla siguen aprobadas.
R06. Encapsular consulta	Carga diferida de relaciones por fila; después select_related.	Mismo JSON; 22 consultas pasan a 1 con 11 solicitudes.
R07. Consolidar reporte	Cuatro conteos separados; después una agregación por estado.	Reportes vacío y no vacío coinciden; una prueba exige una consulta.

Resultados observados

Verificación	Inicial	Refactorizada
Pruebas backend	21 aprobadas	23 aprobadas: mismas 21 y 2 de consultas
Pruebas frontend	6 aprobadas	7 aprobadas: mismas 6 y 1 de reintento
Contrato de API	10 pasos capturados	10 status y JSON idénticos
Listado de 11 solicitudes	22 consultas SQL	1 consulta SQL
Compilación de producción	Aprobada	Aprobada
Migraciones pendientes	Esquema inicial creado	Ninguna diferencia de esquema
Integración HTTP real	No se usa como comparación visual	7 pasos aprobados por proxy Angular-Django

Las pruebas adicionales protegen mejoras concretas; no sustituyen las pruebas conservadas. La igualdad en diez pasos es evidencia de estabilidad de esa muestra, no una demostración de equivalencia para todas las entradas posibles.

Límites del entorno de pruebas

Se usaron dependencias empaquetadas en el ZIP recibido. No se realizó una descarga e instalación limpia con npm ci. Los componentes Angular se probaron en JSDOM con HTTP simulado. La conexión completa se verificó además por HTTP real, sin navegador gráfico.

El entorno bloqueó la consulta RSS de Node utilizada por la telemetría de Angular. Se utilizó una precarga local que sustituye esa estadística únicamente al fallar. El código de la aplicación y el compilador no fueron modificados para ese ajuste. El detalle está en docs/entorno.md.

6. Pruebas y evidencias

Caso	Resultado esperado y verificado
Alta válida	201; datos persistidos; pendiente y media por defecto.
Alta con responsable	201; identificación y nombre del responsable en la respuesta.
Datos inválidos	400; sin crear una solicitud. Incluye texto vacío, longitud, elecciones e identificadores incorrectos.
JSON inválido	400; rechazo de texto mal formado y valores que no son objeto.
Cambio de estado	200; el cambio se conserva al volver a leer desde la base.
Asignar y quitar	200; null quita la asignación sin borrar la solicitud.
Actualización inválida	400; estado anterior conservado.
Filtros combinados	Solo devuelve filas que cumplen título, estado y prioridad.
Solicitud inexistente	404 con error previsto.
CSRF	403 sin token; escritura válida con cookie y cabecera.
Reporte	Conteos correctos con y sin datos; es global.
Interfaz y error de conexión	Muestra lista y reporte, conserva campos ante rechazo, informa errores y permite reintento.

Comandos principales

```
cd backend
python manage.py test mi_app -v 2
python manage.py makemigrations --check --dry-run

# Desde frontend/
npm run build
npm test -- --watch=false
```

Las salidas se guardan en backend_final.txt, frontend_final.txt, build_final.txt y migraciones_final.txt. Los casos detallados están en tests.py, test_rendimiento.py y app.spec.ts. contrato_inicial.json y contrato_refactorizado.json conservan la comparación. integracion_http.json registra los siete pasos reales.

7. Revisión técnica

Se aplicó un checklist con observaciones sobre el código y acciones verificables. La revisión fue asistida por ChatGPT; no se atribuye a un compañero ni a un docente. La revisión personal del alumno está pendiente.

Control	Resultado y evidencia
Responsabilidades	Corregido: vistas pequeñas que delegan operaciones y consultas.
Contrato público	Corregido: mapeo único y diez respuestas comparadas.
Consultas	Corregido: carga de relaciones anticipada y pruebas de límite.
Entradas y errores	Verificado en casos probados: 400, 404, 405 y CSRF.
Tipado y compilación	Verificado: TypeScript y plantillas con comprobación estricta.
Reintento de conexión	Corregido: evento Angular y método cargarCatalogos, con prueba.
Instalación limpia / revisión visual	Pendientes de validación local. No se declaran realizadas.

Feedback y correcciones

CR01. La respuesta de solicitud está repetida en tres puntos. Centralizarla evita contratos distintos al agregar un campo. Se resolvió con solicitud_a_dict y comparación de JSON.

CR02. El listado carga relaciones dentro del bucle y escala con la cantidad de filas. Se resolvió con select_related y una prueba que exige una consulta.

CR03. El botón de reintento usaba un manejador inline y recargaba toda la página. Se reemplazó por el evento Angular y un método de carga comprobable; una prueba adicional verifica el botón.

Criterio de aceptación

Mantener las operaciones acordadas y todas las pruebas iniciales, no modificar el esquema ni el contrato observado y conservar las mejoras de consultas. Los controles incorporados pasan; las limitaciones abiertas permanecen registradas en docs/code_review.md.

8. Conclusiones y cierre pendiente

Conclusión técnica

El caso muestra que una implementación puede resolver las operaciones visibles y aun así presentar deuda técnica. La separación de responsabilidades, el mapeo único y los objetos de parámetros permiten localizar mejor los cambios. Las pruebas aportan evidencia de que esa reorganización conserva las respuestas esperadas.

La mejora medible principal es la eliminación de N+1 en el escenario de listado: 22 consultas pasan a 1 para 11 solicitudes. Esa medición describe cantidad de consultas, no tiempo de respuesta ni rendimiento bajo carga, que no fueron medidos.

Dificultades y límites del proceso

La base recibida todavía no era funcional, por lo que fue necesario construir el prototipo antes del diagnóstico. Se conservan por separado el origen y la versión funcional inicial para no confundir esas etapas. La instalación limpia y la inspección visual quedan pendientes de ejecución local.

El sistema no incluye autenticación, control de permisos, paginación ni restricciones de transición entre estados. No se utiliza en producción. Django 4.2.30 es una dependencia heredada sin soporte; actualizar a una rama mantenida requiere una tarea y verificación separadas.

Reflexión individual a completar por Bagni Mattia

Este apartado requiere experiencia y comprensión propias. Antes de entregar, redactar brevemente: qué concepto resultó más difícil; qué cambio puede explicar y justificar con un ejemplo del código; y qué mejora priorizaría después de ejecutar el sistema. No se inventa una reflexión personal en su nombre.

Entregables y enlaces

Recurso	Estado
Repositorio GitHub	Pendiente de completar
Video de defensa	Pendiente de completar
ZIP de evidencias	Incluye versiones, bundle Git, pruebas, diagnóstico y registro de IA.
Portada y revisión	Completar datos, validar el proyecto y agregar reflexión individual.

El video debe grabarse con voz propia y presentación en cámara. La guía de 12 minutos está en docs/guion_video.md. Los recursos deben estar accesibles al momento de la corrección.

9. IA, anexos y fuentes

Declaración de asistencia

Se utilizó ChatGPT con intervención sustancial para interpretar la consigna, continuar las plantillas aportadas, desarrollar el prototipo funcional, refactorizar, diseñar y ejecutar pruebas, revisar técnicamente y preparar este borrador. No se limita su participación a consultas de teoría o redacción.

El alumno debe revisar el material, ejecutar el sistema y defender las decisiones. No se afirma que esas actividades personales ya hayan ocurrido. docs/registro_ia.md contiene citas reales de intercambios técnicos, resúmenes identificados y acciones de la herramienta. No es una exportación completa; puede complementarse con el chat original si la cátedra lo requiere.

Historial y anexos

Recurso	Contenido
historial_tpi.bundle	Commits reales de esta sesión y tags; se puede clonar con Git.
version_inicial/	Código exportado del tag estado-inicial.
version_refactorizada/	Código final y documentación del proceso.
docs/diagnostico.*	Diez hallazgos y ubicaciones de la versión inicial.
docs/refactorings.md	Relación entre problemas, cambios y verificaciones.
docs/code_review.md	Checklist, observaciones, correcciones y controles pendientes.
docs/*.txt y *.json	Salidas de pruebas, contratos y resultados HTTP.
informe/	PDF provisional y fuente Markdown editable.

Tags: estado-inicial, estadorefactorizado, etapa-diagnostico, etapa-refactoring, etapa-pruebas y etapa-docs. Los commits se identifican como trabajo asistido y no simulan contribuciones anteriores.

Fuentes

Consigna oficial aportada: TP MS2V2 (1).pdf, secciones 4 a 7 y 11. Material complementario aportado: Markdown pegado(7).md.

Django, Writing and running tests: https://docs.djangoproject.com/en/4.2/topics/testing/overview/

Angular, Setting up HttpClient: https://angular.dev/guide/http/setup

Angular, Serving Angular apps for development: https://angular.dev/tools/cli/serve